

Safe Cybersecurity Lab Project (Flask + SQLite)

This project is a local educational cybersecurity lab built with Python and Flask.

It includes:

- Login page
- Admin dashboard
- SQLite database
- Password hashing with bcrypt
- Login attempt logging
- Basic rate limiting logic
- Local-only educational attack simulation concepts

IMPORTANT:

- Use ONLY on localhost or inside your own lab VM.
- Never use against real websites or systems without authorization.

1. Install Requirements

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Create project:

```
mkdir brute-force-lab
cd brute-force-lab
python3 -m venv venv
source venv/bin/activate
```

Install dependencies:

```
pip install flask flask-login flask-limiter bcrypt
```

2. Project Structure

```
brute-force-lab/
|
├─ app.py
├─ init_db.py
├─ users.db
|
├─ templates/
|   ├── login.html
|   └── admin.html
|
├─ static/
|   └── style.css
|
└─ logs/
    └── attempts.log
```

3. Database Initialization

Create a file named `init_db.py`

```
import sqlite3
import bcrypt

conn = sqlite3.connect('users.db')
cur = conn.cursor()

cur.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE,
    password BLOB
)
''')

username = 'admin'
password = 'Admin123!'

hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())

try:
```

```

        cur.execute(
            'INSERT INTO users (username, password) VALUES (?, ?)',
            (username, hashed)
        )
    except:
        pass

    conn.commit()
    conn.close()

    print('Database initialized.')

```

Run:

```
python3 init_db.py
```

4. Main Flask Application

Create a file named `app.py`

```

from flask import Flask, render_template, request, redirect, session
import sqlite3
import bcrypt
import time
from datetime import datetime

app = Flask(__name__)
app.secret_key = 'supersecretkey'

FAILED_ATTEMPTS = {}
LOCKOUT_TIME = 30
MAX_ATTEMPTS = 5

def log_attempt(username, success):
    with open('logs/attempts.log', 'a') as f:
        status = 'SUCCESS' if success else 'FAILED'
        f.write(f'{datetime.now()} - {username} - {status}\n')

@app.route('/')
def home():
    return redirect('/login')

```

```

@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None

    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        if username in FAILED_ATTEMPTS:
            attempts, first_attempt_time = FAILED_ATTEMPTS[username]

            if attempts >= MAX_ATTEMPTS:
                elapsed = time.time() - first_attempt_time

                if elapsed < LOCKOUT_TIME:
                    return f'Account locked. Try again in {int(LOCKOUT_TIME -
elapsed)} seconds.'
                else:
                    FAILED_ATTEMPTS[username] = [0, time.time()]

            conn = sqlite3.connect('users.db')
            cur = conn.cursor()

            cur.execute(
                'SELECT password FROM users WHERE username = ?',
                (username,)
            )

            user = cur.fetchone()
            conn.close()

            if user:
                stored_password = user[0]

                if bcrypt.checkpw(password.encode(), stored_password):
                    session['user'] = username
                    log_attempt(username, True)
                    return redirect('/admin')

            if username not in FAILED_ATTEMPTS:
                FAILED_ATTEMPTS[username] = [1, time.time()]
            else:
                FAILED_ATTEMPTS[username][0] += 1

            log_attempt(username, False)
            error = 'Invalid username or password'

```

```

        return render_template('login.html', error=error)

@app.route('/admin')
def admin():
    if 'user' not in session:
        return redirect('/login')

    return render_template('admin.html', user=session['user'])

@app.route('/logout')
def logout():
    session.pop('user', None)
    return redirect('/login')

if __name__ == '__main__':
    app.run(debug=True)

```

5. Login Page

Create `templates/login.html`

```

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

<div class="container">
    <h1>Cybersecurity Lab Login</h1>

    {% if error %}
        <p class="error">{{ error }}</p>
    {% endif %}

    <form method="POST">
        <input type="text" name="username" placeholder="Username" required>
        <input type="password" name="password" placeholder="Password" required>
        <button type="submit">Login</button>

```

```
        </form>
    </div>

</body>
</html>
```

6. Admin Dashboard

Create `templates/admin.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Admin Panel</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

<div class="container">
    <h1>Admin Dashboard</h1>

    <p>Welcome, {{ user }}</p>

    <h2>Security Features Enabled</h2>

    <ul>
        <li>Password hashing</li>
        <li>Login logging</li>
        <li>Basic lockout protection</li>
        <li>Session management</li>
    </ul>

    <a href="/logout">Logout</a>
</div>

</body>
</html>
```

7. CSS Styling

Create `static/style.css`

```
body {
  font-family: Arial, sans-serif;
  background: #111;
  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.container {
  background: #1e1e1e;
  padding: 30px;
  border-radius: 10px;
  width: 350px;
}

input {
  width: 100%;
  margin-bottom: 10px;
  padding: 10px;
}

button {
  width: 100%;
  padding: 10px;
  cursor: pointer;
}

.error {
  color: red;
}
```

8. Running The Project

Initialize database:

```
python3 init_db.py
```

Run the application:

```
python3 app.py
```

Open browser:

```
http://127.0.0.1:5000
```

Default credentials:

```
Username: admin  
Password: Admin123!
```

9. Educational Security Testing Ideas

You can safely study:

- Weak passwords
- Rate limiting
- Account lockouts
- Logging and monitoring
- Password hashing
- Session management
- Authentication security

You should NOT attack real systems.

10. Future Improvements

After this works, add:

- CAPTCHA
- Dashboard graphs
- JWT authentication
- Docker support
- Nginx reverse proxy
- HTTPS locally
- Admin activity logs
- IP monitoring
- Fake SOC alerts
- SIEM-style dashboard

- Multi-user roles
-

11. Recommended Learning Resources

Learn Flask:

- <https://flask.palletsprojects.com/>

Learn OWASP:

- <https://owasp.org/>

Learn Python:

- <https://www.w3schools.com/python/>

Learn Web Security:

- <https://portswigger.net/web-security>